



DevOps

Assignment 3

Before Starting

- Form a group of 2 students.
- Use the private GitHub repository from Assignment 2.
- Divide tasks equally and document each member's contribution in the README.
- Use proper Git workflow, one branch per task, merge to main via pull requests.
- Do not name folders after team members or roll numbers.

Objective

Provision a complete AWS infrastructure using Terraform only, no manual steps in the AWS Console. You will build a secure network, deploy EC2 instances behind a Load Balancer, store files in S3, auto-scale based on traffic load, and get notified through CloudWatch Alarms when something goes wrong.

Note:

This assignment must represent your own original work. **Copying code from online sources without understanding, sharing code between different groups, and use of AI tools to write Terraform code without learning it yourself will all be treated as plagiarism.** During viva, you will be asked to modify, explain, and debug your own code in real time. If you cannot do this, you will receive zero marks regardless of the submitted report.

Task 1: Custom VPC with Subnetting and NAT Gateway

Requirements

- Create a custom VPC with CIDR block 10.0.0.0/16 and enable DNS support and DNS hostnames.
- Create 2 public subnets in different availability zones (e.g., 10.0.1.0/24 and 10.0.2.0/24).
- Create 2 private subnets in different availability zones (e.g., 10.0.10.0/24 and 10.0.11.0/24).
- Attach an Internet Gateway to the VPC.
- Create a public route table and associate it with both public subnets.
- Allocate an Elastic IP (EIP) for the NAT Gateway.
- Create a NAT Gateway in one of the public subnets and associate the EIP.
- Create a private route table that routes 0.0.0.0/0 traffic through the NAT Gateway.
- Associate the private route table with both private subnets.
- Run terraform state list after applying and show all created resources.
- Demonstrate use of depends_on for the NAT Gateway depending on the Internet Gateway.
- Use terraform output blocks to display VPC ID, subnet IDs, and NAT Gateway ID.
- Run terraform destroy and confirm all resources are deleted from AWS Console.
-

Deliverable: Show the output of terraform plan and terraform apply with screenshots. Show the route tables from the AWS Console and describe how traffic flows from a private instance to the internet.

Task 2: Security Groups and EC2 Instance Deployment

Requirements

- Create a Security Group for a public-facing web server that allows inbound HTTP (port 80), HTTPS (port 443), and SSH (port 22) from your IP only.
- Create a separate Security Group for a private database server that only allows inbound connections on port 3306 from the web server Security Group (not from 0.0.0.0/0).
- Both Security Groups should allow all outbound traffic.
- Launch a t3.micro EC2 instance in the public subnet using the web server Security Group.
- Use a user_data script to install Nginx and create a simple HTML page that displays the instance ID.
- Launch a second t3.micro EC2 instance in the private subnet using the DB Security Group.
- Attach an SSH key pair created via Terraform's aws_key_pair resource.

- SSH into the public EC2 instance using the key pair.
- From the public instance, attempt to SSH into the private instance (bastion host pattern).
- Verify that the private instance cannot be accessed directly from your machine.
- Refactor your instance_type, ami_id, and key_name into Terraform input variables.
- Create a terraform.tfvars file to assign values to these variables.
- Use variable validation to ensure the instance_type is one of: t3.micro, t3.small, t3.medium.
-

Deliverable: Show the Security Group rules in the AWS Console with screenshots. Show the Nginx welcome page accessible via the instance's public IP. Show both instances running in the AWS Console. Include terminal screenshots of the SSH session. Show the error message Terraform produces when an invalid instance_type is provided.

Task 3: S3 Bucket

Requirements:

- Create an S3 bucket with a unique name using a random_id or your roll number as a suffix.
- Enable versioning on the bucket using aws_s3_bucket_versioning.
- Enable server-side encryption using AES-256.
- Block all public access using aws_s3_bucket_public_access_block
- Create an IAM role with a policy that allows an EC2 instance to read and write objects to this S3 bucket only.
- Configure Terraform to store its state file in the S3 bucket you created instead of locally.
- Add a DynamoDB table for state locking to prevent concurrent modifications.
-

Deliverable: Show the bucket settings from the AWS Console with screenshots. Show the .tfstate file stored in S3 from the AWS Console.

Task 4 Auto Scaling Group with CloudWatch Alarms

Requirements

- Create an aws_launch_template with your chosen AMI, instance type t3.micro, the SSH key pair from Task 2, and the web server Security Group.
- Include a user_data script in the launch template that installs the stress-ng tool on boot.
- Create an aws_autoscaling_group that spans both public subnets, with min_size = 1, max_size = 3, and desired_capacity = 1.
- Add a Name tag that propagates to launched instances
- Create an aws_autoscaling_policy for scale-out with adjustment_type = ChangeInCapacity and scaling_adjustment = 1.

- Create an `aws_cloudwatch_metric_alarm` that triggers the scale-out policy when average CPUUtilization $\geq 60\%$ for 2 consecutive periods of 60 seconds.
- Create a second `aws_autoscaling_policy` for scale-in with `scaling_adjustment = -1` and a cooldown of 120 seconds.
- Create a CloudWatch alarm that triggers the scale-in policy when CPUUtilization $\leq 20\%$ for 2 consecutive periods of 60 seconds.
- Set a cooldown of 120 seconds on the scale-out policy to prevent rapid oscillation.
- SSH into a running instance and use the `stress-ng` command to artificially raise CPU utilization above 60%.
- Monitor the ASG activity history and show that a new instance is launched automatically.
- After stopping the stress test, verify that the ASG eventually scales back in to 1 instance.
-

Deliverable: Show the ASG in the AWS Console with the running instances. Show the alarm in the ALARM state in CloudWatch with a screenshot. Show the timeline of scaling events in the AWS Console Activity History tab. Provide terminal screenshots of the stress command, ASG activity history screenshots

Task 5: Elastic Load Balancer with Health Checks

Requirements

- Create an `aws_lb` (ALB) of type application deployed across both public subnets.
- Create a dedicated Security Group for the ALB that allows inbound HTTP (80) from 0.0.0.0/0 and all outbound traffic.
- Modify the EC2 instance Security Group from Task 2 to only allow HTTP traffic from the ALB Security Group, not from the internet directly
- Create an `aws_lb_target_group` with protocol HTTP, port 80, and a health check on path / with `healthy_threshold = 2` and `unhealthy_threshold = 3`.
- Create an `aws_lb_listener` on port 80 that forwards traffic to the target group.
- Attach the Auto Scaling Group to the target group using `aws_autoscaling_attachment`
- Scale the ASG `desired_capacity` to 2 manually using terraform apply.
- Use curl or a browser to send multiple requests to the ALB DNS name and observe which instance responds (hint: display instance ID in the Nginx page).
- Show that requests are being served by different instances, demonstrating load distribution.
-

Deliverable: Show the ALB DNS name from the AWS Console and test it in your browser. Show the target group health check status in the AWS Console. Show at least 2 healthy targets registered. Include a screenshot showing the ALB distributing traffic.

Task 6: Code Reusability with Terraform Modules and Packer

Requirements:

- Create a modules/vpc/ directory containing a self-contained VPC module with inputs for: vpc_cidr, public_subnet_cidrs (list), private_subnet_cidrs (list), and environment (string).
- The module must output: vpc_id, public_subnet_ids, private_subnet_ids, and nat_gateway_id.
- Call the module from your root main.tf and pass appropriate values.
- Create a modules/compute/ module that accepts: ami_id, instance_type, subnet_id, security_group_ids, key_name, and environment.
- The module should create the EC2 instance, output its public_ip and instance_id.
- Create a modules/security/ module for Security Groups that accepts vpc_id and environment, and outputs web_sg_id and db_sg_id.
- Install Packer and write a Packer HCL template (build.pkr.hcl) that: starts from the base Ubuntu AMI, installs Nginx and curl, creates a custom /var/www/html/index.html displaying a welcome message, and saves the result as a custom AMI in your AWS account.
- Run packer build and show the resulting AMI ID in the AWS Console under EC2 > AMIs.
- Update your Terraform compute module to use the custom Packer-built AMI ID

Deliverable: Show terraform plan output that confirms the module is being used correctly. Show how the root module ties the VPC, security, and compute modules together using cross-module references (e.g., module.vpc.vpc_id).

Submission Guidelines

Report Format

- The final report must be submitted in PDF format only. Word documents will not be accepted.
- The report must be neatly formatted with clear headings, subheadings, and figure labels.
- Every screenshot must be followed by a written explanation (do not submit raw output without interpretation).
- Use proper figure captions: e.g., Figure 1: VPC Subnet Architecture in AWS Console.

Code Submission

- Submit all Terraform .tf files and the Packer template in a ZIP archive.
- Organize your files in a logical folder structure (e.g., modules/, environments/, packer/).
- Do NOT include .terraform/ directories, state files, or AWS credentials in your submission.
- Include a README.md file explaining how to initialize and apply your Terraform configuration.

Task	Title	Marks
1	Custom VPC with Subnetting and NAT Gateway	20

2	Security Groups and EC2 Instance Deployment	20
3	S3 Bucket	10
4	Auto Scaling Group with CloudWatch Alarms	20
5	Elastic Load Balancer with Health Checks	20
6	Code Reusability with Modules and Packer	10
	Total	100